Bölüm 5: Listeler ve Demetler

SE 422 · Introduction to Data Science · Konu özeti

Kaynak slayt destesi: week4_Lists_and_Tuples
7 Kasım 2026

İçindekiler

Bölüm 5 · Listeler ve Demetler 3

Bölümün amacı 3

Konu anlatımı 3

Diziler, liste ve tuple (Slayt 1–7) 3

İndeksleme ve len (Slayt 8–9) 3

Değiştirme ve birleştirme (Slayt 10–11) 4

Dilimleme ve in (Slayt 12–13) 4

Liste metotları (Slayt 14–17) 5

Kopyalamak ve listeleri işlemek (Slayt 18–21) 5

Liste üreteçleri (Slayt 22–28) 6

İki boyutlu listeler (Slayt 29–31) 6

Tuple'lar (Slayt 32–34) 6

matplotlib ile grafik (Slayt 35–57) 7

Sık yapılan hatalar 8

Kısa tekrar soruları 8

Cevaplar 8

Bölüm 5 · Listeler ve Demetler

Bölümün amacı

Bu bölüm birden çok değeri tek bir değişkende tutmayı öğretir. Sonunda şunları yapabilmelisin:

- Liste oluşturmak, indekslemek, dilimlemek ve değiştirmek
- Liste metotlarını (`append`, `insert`, `sort`, `remove`, `index`, `reverse`) ve `del`, `min`, `max`, `len` kullanmak
- Listeyi doğru şekilde kopyalamak; liste üreteci (list comprehension) yazmak
- İki boyutlu listelerle çalışmak; tuple ile listenin farkını açıklamak
- matplotlib ile çizgi, çubuk ve pasta grafiği çizmenin temel adımlarını bilmek

Konu anlatımı

Diziler, liste ve tuple (Slayt 1–7)

Dizi (sequence), öğeleri art arda saklanan nesnedir. **Liste** değiştirilebilir (mutable), **tuple** değiştirilemez (immutable) bir dizidir.

Liste köşeli parantezle yazılır ve farklı tipte öğeler tutabilir. `list()` bazı nesneleri (ör. `range`) listeye çevirir; `*` operatörü listeyi tekrarlar.

PYTHON
```python
even_numbers = [2, 4, 6, 8, 10]
info = ['Alicia', 27, 1550.87]
print(info)
print(list(range(5)), [0] * 5)
for n in even_numbers:
    print(n, end=' ')
```

ÇIKTI
```
['Alicia', 27, 1550.87]
[0, 1, 2, 3, 4] [0, 0, 0, 0, 0]
2 4 6 8 10
```

İndeksleme ve len (Slayt 8–9)

İlk elemanın indeksi **0**, son elemanın `len(liste) - 1`'dir. Negatif indeksler sondan sayar (`-1` son eleman). Geçersiz indeks `IndexError` fırlatır.

PYTHON

```python
my_list = [10, 20, 30, 40]
print(my_list[0], my_list[-1], len(my_list))
index = 0
while index < len(my_list):
    print(my_list[index], end=' ')
    index += 1
```

ÇIKTI

```
10 40 4
10 20 30 40
```

Değiştirme ve birleştirme (Slayt 10–11)

Listeler mutable'dır: `liste[i] = yeni` ile eleman değişir. `+` iki listeyi birleştirir, `+=` mevcut listeye ekler.

PYTHON

```python
numbers = [1, 2, 3]
numbers[1] = 99
list1 = [1, 2]
list1 += [3, 4]
print(numbers, list1, [1, 2] + [5])
```

ÇIKTI

```
[1, 99, 3] [1, 2, 3, 4] [1, 2, 5]
```

Dilimleme ve in (Slayt 12–13)

`liste[start:end]` start'tan end'e (**end dahil değil**) bir **kopya** liste döndürür. Başlangıç yazılmazsa 0, bitiş yazılmazsa uzunluk kullanılır; üçüncü sayı adımdır. `in` / `not in` üyeliği sınar.

PYTHON

```python
days = ['Pzt', 'Sal', 'Çar', 'Per', 'Cum']
print(days[1:3], days[:2], days[::2], days[-2:])
print('Cum' in days, 'Cmt' not in days)
```

ÇIKTI

```
['Sal', 'Çar'] ['Pzt', 'Sal'] ['Pzt', 'Çar', 'Cum'] ['Per', 'Cum']
True True
```

Liste metotları (Slayt 14–17)

Metot	Ne yapar?
`append(x)`	sona ekler
`insert(i, x)`	`i` konumuna ekler, sonrakiler kayar
`index(x)`	ilk geçtiği indeks; yoksa `ValueError`
`remove(x)`	ilk geçtiği yeri siler; yoksa `ValueError`
`sort()` / `reverse()`	artan sıralar / ters çevirir

Ayrıca `del liste[i]` belirli indeksi siler; `min` ve `max` en küçük/en büyük değeri verir.

PYTHON

```
names = ['Ali', 'Can']
names.append('Ece')
names.insert(0, 'Zeynep')
print(names)
names.sort()
print(names)
names.remove('Can')
del names[0]
print(names, names.index('Zeynep'), min([4, 1, 7]), max([4, 1, 7]))
```

ÇIKTI

```
['Zeynep', 'Ali', 'Can', 'Ece']
['Ali', 'Can', 'Ece', 'Zeynep']
['Ece', 'Zeynep'] 1 1 7
```

Kopyalamak ve listeleri işlemek (Slayt 18–21)

`list2 = list1` **kopya oluşturmaz**: iki isim aynı listeyi gösterir. Gerçek kopya için elemanları yeni listeye eklemek ya da `[] + list1` yazmak gerekir. Toplam için biriktiricili döngü ya da `sum`, ortalama için `toplam / len` kullanılır. Listeler fonksiyonlara verilebilir ve fonksiyonlardan döndürülebilir; dosyaya her eleman `+ '\n'` ile yazılıp `readlines` ile okunabilir.

PYTHON

```
list1 = [1, 2, 3, 4]
ayni = list1
kopya = [] + list1
list1[0] = 99
print(ayni, kopya)
scores = [2.5, 7.3, 6.5]
print(round(sum(scores) / len(scores), 2))
```

ÇIKTI

```
[99, 2, 3, 4] [1, 2, 3, 4]
5.43
```

Liste üreteçleri (Slayt 22–28)

`[sonuc for oge in liste]` var olan listenin elemanları üzerinden yeni liste kurar. Sonuç ifadesi hesap içerebilir; sona eklenen `if` yalnızca koşulu sağlayan elemanları alır.

PYTHON

```python
list1 = [1, 2, 3, 4]
print([item ** 2 for item in list1])
print([len(s) for s in ['Winken', 'Blinken', 'Nod']])
print([item for item in [1, 12, 2, 20, 3, 15, 4] if item < 10])
```

ÇIKTI

```
[1, 4, 9, 16]
[6, 7, 3]
[1, 2, 3, 4]
```

İki boyutlu listeler (Slayt 29–31)

Elemanları başka listeler olan listedir; satır ve sütun olarak düşünülür. Elemana `liste[satir][sutun]` ile erişilir, genellikle iç içe döngüyle işlenir.

PYTHON

```python
scores = [[70, 80, 90],
          [85, 95, 75]]
print(scores[1][0])
for row in scores:
    for value in row:
        print(value, end=' ')
    print()
```

ÇIKTI

```
85
70 80 90
85 95 75
```

Tuple'lar (Slayt 32–34)

Tuple, oluşturulduktan sonra **değiştirilemeyen** bir dizidir: `(1, 2, 3)`. İndeksleme, dilimleme, `len`, `min`, `max`, `in`, `+`, `*` desteklenir; `append`, `remove`, `insert`, `sort`, `reverse` yoktur. Listeden hızlıdır ve veriyi güvenle korur. `list()` ve `tuple()` birbirine çevirir.

PYTHON
```python
my_tuple = (1, 2, 3, 4, 5)
print(my_tuple[0], len(my_tuple), max(my_tuple), my_tuple[1:3])
print(list(my_tuple), tuple([7, 8]))
```

ÇIKTI
```
1 5 5 (2, 3)
[1, 2, 3, 4, 5] (7, 8)
```

matplotlib ile grafik (Slayt 35–57)

matplotlib standart kütüphanede **değildir**; `pip install matplotlib` ile kurulur ve `import matplotlib.pyplot as plt` ile kullanılır. (Bu uygulamanın ortamında matplotlib kurulu olmadığı için aşağıdaki kodlar çalıştırılmadı; yalnızca gösterim amaçlıdır.)

- **Çizgi grafiği:** `plt.plot(x, y)`; `title`, `xlabel`, `ylabel`, `xlim`/`ylim`, `xticks`/`yticks`, `marker='o'`, `grid(True)`, sonra `plt.show()`.
- **Çubuk grafiği:** `plt.bar(sol_kenarlar, yukseklikler)`; varsayılan genişlik 0.8, `color=('r', 'g', 'b')`.
- **Pasta grafiği:** `plt.pie(degerler, labels=etiketler)`; her dilim değerin toplama oranıdır.

PYTHON
```python
import matplotlib.pyplot as plt
x_coords = [0, 1, 2, 3, 4]
y_coords = [0, 3, 1, 5, 2]
plt.plot(x_coords, y_coords, marker='o')
plt.title('Sales by Year')
plt.xlabel('Year')
plt.ylabel('Sales')
plt.xticks([0, 1, 2, 3, 4], ['2016', '2017', '2018', '2019', '2020'])
plt.grid(True)
plt.show()
```

PYTHON
```python
import matplotlib.pyplot as plt
plt.bar([0, 10, 20], [100, 200, 300], 5, color=('r', 'g', 'b'))
plt.show()
plt.pie([100, 400, 300], labels=['1. çeyrek', '2. çeyrek', '3. çeyrek'])
plt.show()
```

Sık yapılan hatalar

- **Son indeks için `len(liste)` kullanmak** → `IndexError`; son indeks `len(liste) - 1`'dir.

```
PYTHON
l = [3, 5, 7]
print(l[3])
```

```
ÇIKTI
IndexError: list index out of range
```

- `list2 = list1` **ile kopyaladığını sanmak:** biri değişince diğeri de değişir.
- **Listede olmayan öğeyi `remove` / `index` ile aramak** → `ValueError`.
- **Tuple'ı değiştirmeye çalışmak:** `t.append(4)` → `AttributeError`; `t[0] = 9` → `TypeError`.
- `sort()`**'un sonucunu atamak:** `l = l.sort()` listeyi `None` yapar; `sort` listeyi yerinde sıralar ve `None` döndürür.

Kısa tekrar soruları

1 `[10, 20, 30, 40, 50][1:4]` nedir? `[...][-2:]` nedir?

2 `a = [1, 2]`, `b = a`, `b.append(3)` sonrası `a` nedir?

3 `[x * 2 for x in range(5) if x % 2 == 1]` nedir?

4 `m = [[1, 2, 3], [4, 5, 6]]` için `m[1][2]` nedir?

5 Tuple'ın listeye göre iki avantajını say.

6 matplotlib neden ayrıca kurulmalıdır?

Cevaplar

1 `[20, 30, 40]` ve `[40, 50]`.

2 `[1, 2, 3]`; `b` ve `a` aynı listeyi gösterir.

3 `[2, 6]`

4 `6`

5 İşlenmesi daha hızlıdır; değiştirilemediği için veriyi güvenle korur (ayrıca bazı işlemler tuple gerektirir).

6 Standart kütüphanenin parçası değildir.